KPR Institute of Engineering and Technology (Autonomous)

Department of CSE | Academic Year 2026-2027

U21CS501 - Web Technology

Assignment II - Solutions

Name	Midhun P
Roll Number	24CS113
Year / Sem / Section	III / V / A
Issue / Submission Date	07.10.2026 / 20.10.2026
Maximum Marks	40
GitHub Repository	https://github.com/<your-username>/student-portal (replace)

Contents

• Project structure and how to run

• Database schema script (shared by PHP and Servlet modules)

• Q1 - HTML form, CSS and JavaScript validation (8 marks)

• Q2 - PHP + MySQL CRUD with file upload (16 marks)

• Q3 - Servlet + JDBC + JSP portal with sessions (16 marks)

• Output screenshots (to be pasted) and conclusion

0. Project Structure and Setup

Folder layout

```
student_portal/                          <- PHP + front-end module (htdocs/student_portal in XAMPP)
  register.html                          Q1 form
  css/style.css                          Q1 styles
  js/validate.js                         Q1 validation
  php/index.php add.php edit.php view.php delete.php       Q2 pages
  php/includes/db.php functions.php header.php footer.php form_fields.php
  uploads/                               shared upload folder (profile images, ID proof, certificates)
  uploads/.htaccess                      "php_flag engine off"  (never execute uploaded files)
  db/schema.sql                          database script

StudentPortalWeb/                        <- Servlet module (Tomcat 9, Java 8+, javax.servlet)
  src/main/java/com/kpr/portal/
      DBUtil.java Student.java StudentFile.java LoginServlet.java LogoutServlet.java
      AuthFilter.java StudentListServlet.java FileServlet.java
  src/main/webapp/login.jsp
  src/main/webapp/WEB-INF/web.xml
  src/main/webapp/WEB-INF/views/students.jsp
  src/main/webapp/WEB-INF/lib/   mysql-connector-j.jar, jstl-1.2.jar
```

• Both modules use the **same MySQL database** (`student_portal`) and the **same uploads folder**. PHP saves files and metadata; the Servlet reads the metadata and streams the files.

• Run: import `db/schema.sql` in phpMyAdmin; start Apache + MySQL; open `http://localhost/student_portal/register.html`. Deploy the Servlet WAR on Tomcat and open `http://localhost:8080/StudentPortalWeb/` (login: admin / Admin@123).

• Set `UPLOAD_DIR` in `web.xml` to the absolute path of `student_portal/uploads`. For Tomcat 10+ replace `javax.` with `jakarta.` and use the jakarta JSTL URI.

DB Schema Script (db/schema.sql)

db/schema.sql

```
CREATE DATABASE IF NOT EXISTS student_portal CHARACTER SET utf8mb4;
USE student_portal;

-- Login accounts used by the Servlet portal
CREATE TABLE users (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  username      VARCHAR(50) NOT NULL UNIQUE,
  password_hash CHAR(64)    NOT NULL                -- SHA-256 hex digest
);

-- Student master table
CREATE TABLE students (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  name          VARCHAR(100) NOT NULL,
  roll_no       VARCHAR(20)  NOT NULL UNIQUE,
  email         VARCHAR(120) NOT NULL,
  course        VARCHAR(60)  NOT NULL,
  password_hash VARCHAR(255) NULL,                  -- bcrypt hash (registration form)
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Metadata of every uploaded file: one student -> many files
CREATE TABLE student_files (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  student_id    INT NOT NULL,
  file_type     ENUM('profile','id_proof','certificate') NOT NULL,
  original_name VARCHAR(255) NOT NULL,
  file_path     VARCHAR(255) NOT NULL,              -- unique stored name inside uploads/
  mime_type     VARCHAR(100) NOT NULL,
  file_size     INT NOT NULL,
  uploaded_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- upload date
  CONSTRAINT fk_student FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE
);

INSERT INTO users (username, password_hash) VALUES ('admin', SHA2('Admin@123', 256));
```

Question 1 - Student Registration Form (8 Marks, CO3)

Approach. `register.html` holds semantic HTML5 form controls (labels bound to inputs, *select*, *file*). `style.css` gives a card layout, a 2-column CSS grid that collapses to one column under 600 px (responsive), focus/valid/invalid states and styled error text. `validate.js` validates every field on blur, on input and on submit: required fields, email regex, roll-number pattern, password strength (8+ chars, upper, lower, digit, special) with a live strength meter, confirm-password match and file type/size. The form posts to `php/add.php` (Q2), which repeats all checks on the server.

register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0"> <!-- responsive design -->
  <title>Student Registration</title>
  <link rel="stylesheet" href="css/style.css">
</head>
<body>
<main class="card">
  <h1>Student Registration</h1>
  <p class="sub">Fields marked * are required</p>

  <!-- novalidate: browser pop-ups are off; validate.js shows styled messages instead -->
  <form id="regForm" action="php/add.php" method="post" enctype="multipart/form-data" novalidate>
    <div class="grid">
      <div class="field">
        <label for="name">Full Name *</label>
        <input type="text" id="name" name="name" placeholder="Arun Kumar">
        <small class="error"></small>
      </div>
      <div class="field">
        <label for="roll_no">Roll Number *</label>
        <input type="text" id="roll_no" name="roll_no" placeholder="21CS0101">
        <small class="error"></small>
      </div>
      <div class="field full">
        <label for="email">Email *</label>
        <input type="email" id="email" name="email" placeholder="arun@example.com">
        <small class="error"></small>
      </div>
      <div class="field full">
        <label for="course">Course *</label>
        <select id="course" name="course">
          <option value="">-- Select course --</option>
          <option>B.E. CSE</option><option>B.E. ECE</option><option>B.E. EEE</option>
          <option>B.Tech IT</option><option>B.E. Mechanical</option>
        </select>
        <small class="error"></small>
      </div>
      <div class="field">
        <label for="password">Password *</label>
        <input type="password" id="password" name="password" autocomplete="new-password">
        <div class="meter"><span id="bar"></span></div>
        <small id="strengthText"></small>
        <small class="error"></small>
      </div>
      <div class="field">
        <label for="confirm_password">Confirm Password *</label>
        <input type="password" id="confirm_password" name="confirm_password" autocomplete="new-password">
        <small class="error"></small>
      </div>
      <div class="field">
        <label for="profile_image">Profile Image * (JPG/PNG, max 2 MB)</label>
        <input type="file" id="profile_image" name="profile_image" accept=".jpg,.jpeg,.png">
        <small class="error"></small>
      </div>
      <div class="field">
        <label for="id_proof">ID Proof (PDF/JPG/PNG, max 5 MB)</label>
        <input type="file" id="id_proof" name="id_proof" accept=".pdf,.jpg,.jpeg,.png">
        <small class="error"></small>
      </div>
    </div>
    <button type="submit">Register</button>
  </form>
</main>
<script src="js/validate.js"></script>
</body>
</html>
```

css/style.css

```
:root { --primary:#1d4ed8; --danger:#dc2626; --ok:#16a34a; --bg:#eef2ff; --text:#1f2937; }
* { box-sizing: border-box; }
body { margin:0; padding:24px; display:flex; justify-content:center; background:var(--bg);
       color:var(--text); font-family:"Segoe UI", Arial, sans-serif; }
.card { width:100%; max-width:640px; padding:28px; background:#fff; border-radius:12px;
        box-shadow:0 8px 24px rgba(0,0,0,.12); }
h1 { margin:0; color:var(--primary); }
.sub { margin:4px 0 20px; color:#6b7280; }

/* two-column grid; .full spans both columns */
.grid { display:grid; grid-template-columns:1fr 1fr; gap:16px; }
.full { grid-column:1 / -1; }
.field { display:flex; flex-direction:column; }
label { margin-bottom:6px; font-size:.9rem; font-weight:600; }
input, select { padding:10px 12px; font-size:1rem; border:1.5px solid #d1d5db; border-radius:8px; }
input:focus, select:focus { outline:none; border-color:var(--primary);
                            box-shadow:0 0 0 3px rgba(29,78,216,.15); }

/* validation states (class toggled by validate.js) */
.field.invalid input, .field.invalid select { border-color:var(--danger); background:#fef2f2; }
.field.valid input, .field.valid select     { border-color:var(--ok); }
.error { min-height:1.1em; margin-top:4px; font-size:.8rem; color:var(--danger); }

/* password strength meter */
.meter { height:6px; margin-top:6px; background:#e5e7eb; border-radius:4px; overflow:hidden; }
.meter span { display:block; width:0; height:100%; transition:width .3s, background .3s; }
#strengthText { font-size:.75rem; color:#6b7280; }

button { width:100%; margin-top:22px; padding:12px; font-size:1rem; color:#fff;
         background:var(--primary); border:0; border-radius:8px; cursor:pointer; }
button:hover { background:#1e40af; }

/* responsive: single column on phones */
@media (max-width:600px) { .grid { grid-template-columns:1fr; } .card { padding:18px; } }
```

js/validate.js

```
(function () {
  'use strict';
  const form = document.getElementById('regForm');
  const MB = 1024 * 1024;

  // Password score 0-5: one point per satisfied rule
  function strength(p) {
    return [p.length >= 8, /[a-z]/.test(p), /[A-Z]/.test(p), /\d/.test(p), /[^A-Za-z0-9]/.test(p)]
           .filter(Boolean).length;
  }

  // File check: allowed extensions + maximum size (a file is mandatory only when "required")
  function fileCheck(file, exts, max, required) {
    if (!file) return required ? 'Please choose a file.' : '';
    const ext = file.name.split('.').pop().toLowerCase();
    if (!exts.includes(ext)) return 'Allowed types: ' + exts.join(', ').toUpperCase() + '.';
    if (file.size > max) return 'File must be smaller than ' + (max / MB) + ' MB.';
    return '';
  }

  // Each validator returns '' when valid, otherwise the error message
  const validators = {
    name:      v => /^[A-Za-z][A-Za-z .]{2,49}$/.test(v) ? '' : 'Enter 3-50 letters (spaces/dots allowed).',
    roll_no:   v => /^\d{2}[A-Za-z]{2,3}\d{3,4}$/.test(v) ? '' : 'Roll number format: 21CS0101.',
    email:     v => /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/.test(v) ? '' : 'Enter a valid email address.',
    course:    v => v ? '' : 'Please select a course.',
    password: v => strength(v) === 5 ? '' : 'Min 8 chars with upper, lower, digit and special character.',
    confirm_password: v => v === form.password.value ? '' : 'Passwords do not match.',
    profile_image: (v, f) => fileCheck(f, ['jpg', 'jpeg', 'png'], 2 * MB, true),
    id_proof:       (v, f) => fileCheck(f, ['pdf', 'jpg', 'jpeg', 'png'], 5 * MB, false)
  };

  // Validate one control, update its styling and message; returns true when valid
  function check(el) {
    const box = el.closest('.field');
    const isFile = el.type === 'file';
    const isPwd = el.type === 'password';
    const value = isFile ? '' : (isPwd ? el.value : el.value.trim());
    let msg;
    if (!isFile && value === '') msg = 'This field is required.';
    else msg = validators[el.name](value, isFile ? el.files[0] : null);
    box.classList.toggle('invalid', msg !== '');
    box.classList.toggle('valid', msg === '');
    box.querySelector('.error').textContent = msg;
    return msg === '';
  }

  const fields = Array.from(form.elements).filter(el => el.name && validators[el.name]);
  fields.forEach(el => {
    el.addEventListener('blur', () => check(el));
    el.addEventListener('change', () => check(el));
    el.addEventListener('input', () => {                      // live re-check once an error is shown
      if (el.closest('.field').classList.contains('invalid')) check(el);
    });
  });

  // Live password-strength meter
  const bar = document.getElementById('bar'), text = document.getElementById('strengthText');
  const colours = ['#dc2626', '#dc2626', '#f59e0b', '#f59e0b', '#84cc16', '#16a34a'];
  const labels = ['', 'Very weak', 'Weak', 'Fair', 'Good', 'Strong'];
  form.password.addEventListener('input', () => {
    const s = strength(form.password.value);
    bar.style.width = (s * 20) + '%';
    bar.style.background = colours[s];
    text.textContent = form.password.value ? labels[s] : '';
  });

  // Block submission when any field is invalid
  form.addEventListener('submit', e => {
    const results = fields.map(check);                        // check ALL fields (no short-circuit)
    if (results.includes(false)) {
      e.preventDefault();
      fields[results.indexOf(false)].focus();                 // jump to first error
    }
  });
})();
```

Question 2 - PHP + MySQL: CRUD and File Upload (16 Marks, CO4)

Approach. PDO with prepared statements is used for all queries (prevents SQL injection) and output is escaped with `h()` (prevents XSS). `add.php` creates a student, `index.php` lists, `view.php` shows details and uploaded files, `edit.php` updates and `delete.php` removes a record.

• **Upload handling:** three inputs - profile image, ID proof, certificate - each with its own allowed extensions, real MIME type check (`finfo`) and size limit (2 MB / 5 MB).

• **No overwriting:** files are stored as `random_hex_timestamp.ext` and a loop guarantees the name does not already exist. The original name is kept only in the database.

• **Metadata:** file path, type, MIME, size and upload date are saved in `student_files` linked by `student_id`.

• **Error handling:** all validation runs before the INSERT; the insert and uploads run in one transaction; duplicate roll numbers are caught (SQLSTATE 23000); deleting a student also deletes its files from disk.

php/includes/db.php

```
<?php
// Database connection (PDO) and shared paths
define('UPLOAD_DIR', dirname(__DIR__, 2) . '/../uploads/');      // physical folder: student_portal/uploads/
define('UPLOAD_URL', '../uploads/');                             // URL used by <img> / <a> in php/ pages

try {
    $pdo = new PDO('mysql:host=localhost;dbname=student_portal;charset=utf8mb4', 'root', '', [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,    // throw exceptions on SQL errors
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,                     // real prepared statements
    ]);
} catch (PDOException $e) {
    http_response_code(500);
    exit('Database connection failed.');                           // do not leak details to the user
}
```

php/includes/functions.php

```php
<?php
const COURSES = ['B.E. CSE', 'B.E. ECE', 'B.E. EEE', 'B.Tech IT', 'B.E. Mechanical'];

// Per-field upload rules: allowed extensions, allowed MIME types, max size in bytes
const UPLOAD_RULES = [
    'profile_image' => ['type' => 'profile', 'max' => 2 * 1048576,
                        'ext' => ['jpg', 'jpeg', 'png'], 'mime' => ['image/jpeg', 'image/png']],
    'id_proof'      => ['type' => 'id_proof', 'max' => 5 * 1048576,
                        'ext' => ['pdf', 'jpg', 'jpeg', 'png'],
                        'mime' => ['application/pdf', 'image/jpeg', 'image/png']],
    'certificate'   => ['type' => 'certificate', 'max' => 5 * 1048576,
                        'ext' => ['pdf', 'jpg', 'jpeg', 'png'],
                        'mime' => ['application/pdf', 'image/jpeg', 'image/png']],
];

// Escape output (XSS protection)
function h($s) { return htmlspecialchars((string)$s, ENT_QUOTES, 'UTF-8'); }

// Read and trim the text fields of the form
function collect_input(): array {
    $d = [];
    foreach (['name', 'roll_no', 'email', 'course', 'password'] as $k) {
        $d[$k] = trim($_POST[$k] ?? '');
    }
    return $d;
}

// Server-side validation of text fields; returns a list of error messages
function validate_student(array $d): array {
    $e = [];
    if (!preg_match('/^[A-Za-z][A-Za-z .]{2,49}$/', $d['name']))      $e[] = 'Name must be 3-50 letters.';
    if (!preg_match('/^\d{2}[A-Za-z]{2,3}\d{3,4}$/', $d['roll_no'])) $e[] = 'Invalid roll number (21CS0101).';
    if (!filter_var($d['email'], FILTER_VALIDATE_EMAIL))              $e[] = 'Invalid email address.';
    if (!in_array($d['course'], COURSES, true))                       $e[] = 'Select a valid course.';
    if ($d['password'] !== '' &&
        !preg_match('/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{8,}$/',  $d['password'])) {
        $e[] = 'Password needs 8+ chars with upper, lower, digit and special character.';
    }
    return $e;
}

// Validate one uploaded file; returns an error message or null (no file chosen is not an error)
function validate_upload(string $field): ?string {
    if (!isset($_FILES[$field]) || $_FILES[$field]['error'] === UPLOAD_ERR_NO_FILE) return null;
    $f = $_FILES[$field];
    $r = UPLOAD_RULES[$field];
    if (in_array($f['error'], [UPLOAD_ERR_INI_SIZE, UPLOAD_ERR_FORM_SIZE], true) || $f['size'] > $r['max']) {
        return "$field: file is larger than " . ($r['max'] / 1048576) . " MB.";
    }
    if ($f['error'] !== UPLOAD_ERR_OK) return "$field: upload failed (error code {$f['error']}).";
    $ext  = strtolower(pathinfo($f['name'], PATHINFO_EXTENSION));
    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($f['tmp_name']);     // real content type, not browser's claim
    if (!in_array($ext, $r['ext'], true) || !in_array($mime, $r['mime'], true)) {
        return "$field: only " . strtoupper(implode('/', $r['ext'])) . " files are allowed.";
    }
    return null;
}

// Move a (validated) upload into uploads/ and store its metadata in MySQL
function save_upload(PDO $pdo, int $sid, string $field): void {
    if (!isset($_FILES[$field]) || $_FILES[$field]['error'] !== UPLOAD_ERR_OK) return;
    $f = $_FILES[$field];
    $r = UPLOAD_RULES[$field];
    $ext  = strtolower(pathinfo($f['name'], PATHINFO_EXTENSION));
    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($f['tmp_name']);
    if (!is_dir(UPLOAD_DIR)) mkdir(UPLOAD_DIR, 0755, true);

    do {                                                              // unique name => never overwrites
        $stored = bin2hex(random_bytes(8)) . '_' . time() . '.' . $ext;
    } while (file_exists(UPLOAD_DIR . $stored));

    if (!move_uploaded_file($f['tmp_name'], UPLOAD_DIR . $stored)) {
        throw new RuntimeException("Could not store $field.");
    }
    if ($r['type'] === 'profile') delete_files($pdo, $sid, 'profile'); // keep only one profile image

    $pdo->prepare('INSERT INTO student_files
                   (student_id, file_type, original_name, file_path, mime_type, file_size)
                   VALUES (?, ?, ?, ?, ?, ?)')
        ->execute([$sid, $r['type'], basename($f['name']), $stored, $mime, $f['size']]);
}

// Delete a student's files (all, or one type) from disk and from the metadata table
```

```
function delete_files(PDO $pdo, int $sid, ?string $type = null): void {
    $sql = 'SELECT id, file_path FROM student_files WHERE student_id = ?' . ($type ? ' AND file_type = ?' : '');
    $st = $pdo->prepare($sql);
    $st->execute($type ? [$sid, $type] : [$sid]);
    foreach ($st->fetchAll() as $row) {
        @unlink(UPLOAD_DIR . basename($row['file_path']));
        $pdo->prepare('DELETE FROM student_files WHERE id = ?')->execute([$row['id']]);
    }
}
```

php/includes/header.php and footer.php

```
<?php /* header.php - $title must be set before including */ ?>
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title><?= h($title) ?></title>
<style>
  body { font-family:"Segoe UI",Arial,sans-serif; background:#eef2ff; margin:0; padding:20px; color:#1f2937; }
  .wrap { max-width:900px; margin:auto; background:#fff; padding:24px; border-radius:12px;
          box-shadow:0 6px 18px rgba(0,0,0,.1); }
  table { width:100%; border-collapse:collapse; }
  th, td { padding:8px; border-bottom:1px solid #e5e7eb; text-align:left; }
  th { background:#1d4ed8; color:#fff; }
  a.btn, button { background:#1d4ed8; color:#fff; padding:6px 12px; border:0; border-radius:6px;
                  text-decoration:none; cursor:pointer; font-size:.9rem; display:inline-block; }
  .red { background:#dc2626; }
  .err { background:#fee2e2; color:#991b1b; padding:10px 16px; border-radius:8px; margin-bottom:12px; }
  .ok  { background:#dcfce7; color:#166534; padding:10px 16px; border-radius:8px; margin-bottom:12px; }
  label { display:block; margin:12px 0 4px; font-weight:600; }
  input, select { width:100%; padding:8px; border:1px solid #d1d5db; border-radius:6px; box-sizing:border-box; }
  .avatar { width:140px; height:140px; object-fit:cover; border-radius:50%; border:3px solid #1d4ed8; }
  .thumb { max-width:120px; max-height:90px; border:1px solid #d1d5db; border-radius:4px; }
  @media (max-width:600px) { .hide-sm { display:none; } }
</style></head><body><div class="wrap">

<?php /* footer.php */ ?>
</div></body></html>
```

php/includes/form_fields.php

```
<?php /* Shared by add.php and edit.php. Expects $s (current values) and $showPassword (bool) */ ?>
<label>Full Name *</label>
<input name="name" value="<?= h($s['name']) ?>" required>
<label>Roll Number *</label>
<input name="roll_no" value="<?= h($s['roll_no']) ?>" required>
<label>Email *</label>
<input type="email" name="email" value="<?= h($s['email']) ?>" required>
<label>Course *</label>
<select name="course" required>
  <option value="">-- Select --</option>
  <?php foreach (COURSES as $c): ?>
    <option <?= $s['course'] === $c ? 'selected' : '' ?>><?= h($c) ?></option>
  <?php endforeach; ?>
</select>
<?php if ($showPassword): ?>
  <label>Password (optional, 8+ chars with upper, lower, digit, special)</label>
  <input type="password" name="password" autocomplete="new-password">
<?php endif; ?>
<label>Profile Image (JPG/PNG, max 2 MB)</label>
<input type="file" name="profile_image" accept=".jpg,.jpeg,.png">
<label>ID Proof (PDF/JPG/PNG, max 5 MB)</label>
<input type="file" name="id_proof" accept=".pdf,.jpg,.jpeg,.png">
<label>Certificate (PDF/JPG/PNG, max 5 MB)</label>
<input type="file" name="certificate" accept=".pdf,.jpg,.jpeg,.png">
```

php/index.php (View all students)

```
<?php
require 'includes/db.php';
require 'includes/functions.php';
$title = 'Student Records';

// one query: students + number of uploaded files for each
$rows = $pdo->query('SELECT s.*, (SELECT COUNT(*) FROM student_files f WHERE f.student_id = s.id) AS files
                     FROM students s ORDER BY s.roll_no')->fetchAll();
include 'includes/header.php';
?>
<h2>Student Records</h2>
<?php if (isset($_GET['msg'])): ?><div class="ok"><?= h($_GET['msg']) ?></div><?php endif; ?>
<p><a class="btn" href="add.php">+ Add Student</a></p>
<table>
  <tr><th>Roll  No</th><th>Name</th><th  class="hide-sm">Course</th><th>Files</th><th>Actions</th></tr>
  <?php foreach ($rows as $r): ?>
  <tr>
    <td><?= h($r['roll_no']) ?></td>
    <td><?= h($r['name']) ?></td>
    <td class="hide-sm"><?= h($r['course']) ?></td>
    <td><?= (int)$r['files'] ?></td>
    <td>
      <a class="btn" href="view.php?id=<?= (int)$r['id'] ?>">View</a>
      <a class="btn" href="edit.php?id=<?= (int)$r['id'] ?>">Edit</a>
      <form method="post" action="delete.php" style="display:inline"
            onsubmit="return confirm('Delete this student and all uploaded files?');">
        <input type="hidden" name="id" value="<?= (int)$r['id'] ?>">
        <button class="red">Delete</button>
      </form>
    </td>
  </tr>
  <?php endforeach; ?>
  <?php if (!$rows): ?><tr><td colspan="5">No students yet.</td></tr><?php endif; ?>
</table>
<?php include 'includes/footer.php'; ?>
```

php/add.php (Create)

```php
<?php
require 'includes/db.php';
require 'includes/functions.php';
$title = 'Add Student';
$errors = [];
$s = ['name' => '', 'roll_no' => '', 'email' => '', 'course' => ''];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $d = collect_input();
    $s = $d;
    $errors = validate_student($d);                                    // 1. validate text fields

    foreach (array_keys(UPLOAD_RULES) as $field) {                     // 2. validate every upload
        if ($msg = validate_upload($field)) $errors[] = $msg;
    }
    if (!isset($_FILES['profile_image']) || $_FILES['profile_image']['error'] === UPLOAD_ERR_NO_FILE) {
        $errors[] = 'Profile image is required.';
    }

    if (!$errors) {                                                    // 3. save only if everything is valid
        try {
            $pdo->beginTransaction();
            $pdo->prepare('INSERT INTO students (name, roll_no, email, course, password_hash)
                           VALUES (?, ?, ?, ?, ?)')
                ->execute([$d['name'], $d['roll_no'], $d['email'], $d['course'],
                           $d['password'] !== '' ? password_hash($d['password'], PASSWORD_DEFAULT) : null]);
            $sid = (int)$pdo->lastInsertId();
            foreach (array_keys(UPLOAD_RULES) as $field) save_upload($pdo, $sid, $field);
            $pdo->commit();
            header("Location: view.php?id=$sid&msg=Student+added");
            exit;
        } catch (Exception $ex) {
            if ($pdo->inTransaction()) $pdo->rollBack();
            $errors[] = ($ex instanceof PDOException && $ex->getCode() == '23000')
                      ? 'Roll number already exists.' : 'Could not save the student. Try again.';
        }
    }
}
$showPassword = true;
include 'includes/header.php';
?>
<h2>Add Student</h2>
<?php foreach ($errors as $e): ?><div class="err"><?= h($e) ?></div><?php endforeach; ?>
<form method="post" enctype="multipart/form-data">
  <?php include 'includes/form_fields.php'; ?>
  <p><button>Save</button> <a class="btn red" href="index.php">Cancel</a></p>
</form>
<?php include 'includes/footer.php'; ?>
```

php/view.php (Read + display uploaded files)

```
<?php
require 'includes/db.php';
require 'includes/functions.php';
$id = (int)($_GET['id'] ?? 0);

$st = $pdo->prepare('SELECT * FROM students WHERE id = ?');
$st->execute([$id]);
$s = $st->fetch();
if (!$s) { http_response_code(404); exit('Student not found.'); }

$fs = $pdo->prepare('SELECT * FROM student_files WHERE student_id = ? ORDER BY uploaded_at');
$fs->execute([$id]);
$files = $fs->fetchAll();
$profile = null;                                        // pick the profile image (if any)
foreach ($files as $f) if ($f['file_type'] === 'profile') $profile = $f;

$title = 'Student Details';
include 'includes/header.php';
?>
<h2>Student Details</h2>
<?php if (isset($_GET['msg'])): ?><div class="ok"><?= h($_GET['msg']) ?></div><?php endif; ?>
<?php if ($profile): ?>
  <img class="avatar" src="<?= UPLOAD_URL . h($profile['file_path']) ?>" alt="Profile image">
<?php endif; ?>
<table>
  <tr><th>Name</th><td><?= h($s['name']) ?></td></tr>
  <tr><th>Roll No</th><td><?= h($s['roll_no']) ?></td></tr>
  <tr><th>Email</th><td><?= h($s['email']) ?></td></tr>
  <tr><th>Course</th><td><?= h($s['course']) ?></td></tr>
  <tr><th>Registered</th><td><?= h($s['created_at']) ?></td></tr>
</table>

<h3>Uploaded Files</h3>
<table>
  <tr><th>Type</th><th>File</th><th>Size</th><th>Uploaded</th><th>Preview</th></tr>
  <?php foreach ($files as $f): ?>
  <tr>
    <td><?= h($f['file_type']) ?></td>
    <td><a href="<?= UPLOAD_URL . h($f['file_path']) ?>" target="_blank"><?= h($f['original_name']) ?></a></td>
    <td><?= round($f['file_size'] / 1024, 1) ?> KB</td>
    <td><?= h($f['uploaded_at']) ?></td>
    <td>
      <?php if (strpos($f['mime_type'], 'image/') === 0): ?>
        <img class="thumb" src="<?= UPLOAD_URL . h($f['file_path']) ?>" alt="">
      <?php else: ?>PDF<?php endif; ?>
    </td>
  </tr>
  <?php endforeach; ?>
  <?php if (!$files): ?><tr><td colspan="5">No files uploaded.</td></tr><?php endif; ?>
</table>
<p><a class="btn" href="edit.php?id=<?= $id ?>">Edit</a> <a class="btn" href="index.php">Back</a></p>
<?php include 'includes/footer.php'; ?>
```

php/edit.php (Update)

```
<?php
require 'includes/db.php';
require 'includes/functions.php';
$id = (int)($_GET['id'] ?? 0);

$st = $pdo->prepare('SELECT * FROM students WHERE id = ?');
$st->execute([$id]);
$s = $st->fetch();
if (!$s) { http_response_code(404); exit('Student not found.'); }

$errors = [];
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $d = collect_input();
    $s = array_merge($s, $d);                                          // keep typed values in the form
    $errors = validate_student($d);
    foreach (array_keys(UPLOAD_RULES) as $field) {
        if ($msg = validate_upload($field)) $errors[] = $msg;
    }
    if (!$errors) {
        try {
            $pdo->beginTransaction();
            $pdo->prepare('UPDATE students SET name = ?, roll_no = ?, email = ?, course = ? WHERE id = ?')
                ->execute([$d['name'], $d['roll_no'], $d['email'], $d['course'], $id]);
            // a new profile image replaces the old one; documents are added alongside existing ones
            foreach (array_keys(UPLOAD_RULES) as $field) save_upload($pdo, $id, $field);
            $pdo->commit();
            header("Location: view.php?id=$id&msg=Student+updated");
            exit;
        } catch (Exception $ex) {
            if ($pdo->inTransaction()) $pdo->rollBack();
            $errors[] = ($ex instanceof PDOException && $ex->getCode() == '23000')
                        ? 'Roll number already exists.' : 'Could not update the student.';
        }
    }
}
$title = 'Edit Student';
$showPassword = false;
include 'includes/header.php';
?>
<h2>Edit Student</h2>
<?php foreach ($errors as $e): ?><div class="err"><?= h($e) ?></div><?php endforeach; ?>
<form method="post" enctype="multipart/form-data">
  <?php include 'includes/form_fields.php'; ?>
  <p><button>Update</button> <a class="btn red" href="view.php?id=<?= $id ?>">Cancel</a></p>
</form>
<?php include 'includes/footer.php'; ?>
```

php/delete.php (Delete)

```
<?php
require 'includes/db.php';
require 'includes/functions.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {            // deletion only through POST (not by a link/crawler)
    header('Location: index.php');
    exit;
}
$id = (int)($_POST['id'] ?? 0);
delete_files($pdo, $id);                                // remove physical files and metadata rows
$pdo->prepare('DELETE FROM students WHERE id = ?')->execute([$id]);
header('Location: index.php?msg=Student+deleted');
exit;
```

uploads/.htaccess

```
# Uploaded files must never be executed as scripts
php_flag engine off
Options -Indexes
```

Question 3 - Servlet + JDBC + JSP Student Portal (16 Marks, CO5)

Approach (MVC). `LoginServlet` authenticates and creates a session. `AuthFilter` guards `/students` and `/file`. `StudentListServlet` (controller) reads students and their file metadata through JDBC and forwards to `students.jsp` (view) placed under WEB-INF so it cannot be opened directly. The JSP loops over students with JSTL and uses conditions to show a photo or placeholder and the list of documents. `FileServlet` streams the stored image or document: the browser's `<img src="file?id=7">` calls the servlet, which finds the file path in MySQL and copies the bytes from the uploads folder to the response.

• **Security:** the file path comes from the database, never from the request; the resolved path is checked to stay inside the upload folder (blocks path traversal); the session ID is regenerated on login (blocks session fixation); logout invalidates the session; no-store cache headers stop the Back button from showing pages after logout; JSP output uses `c:out` (XSS-safe).

• **JDBC:** PreparedStatement and try-with-resources everywhere, so connections are always closed.

WEB-INF/web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="4.0">
  <display-name>StudentPortalWeb</display-name>

  <!-- Folder where PHP stored the uploaded files (absolute path) -->
  <context-param>
    <param-name>UPLOAD_DIR</param-name>
    <param-value>C:/xampp/htdocs/student_portal/uploads</param-value>
  </context-param>

  <session-config>
    <session-timeout>15</session-timeout>                    <!-- minutes of inactivity -->
    <cookie-config><http-only>true</http-only></cookie-config>
  </session-config>

  <welcome-file-list><welcome-file>login.jsp</welcome-file></welcome-file-list>
</web-app>
```

DBUtil.java

```
package com.kpr.portal;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/** Single place that knows how to connect to MySQL through JDBC. */
public class DBUtil {
    private static final String URL =
        "jdbc:mysql://localhost:3306/student_portal?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASS = "";

    static {                                                  // load the driver once
        try { Class.forName("com.mysql.cj.jdbc.Driver"); }
        catch (ClassNotFoundException e) { throw new ExceptionInInitializerError(e); }
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

Student.java

```
package com.kpr.portal;

import java.util.ArrayList;
import java.util.List;

/** A student together with the metadata of the files uploaded for that student. */
public class Student {
    private final int id;
    private final String name, rollNo, email, course;
    private int profileFileId;                                    // 0 = no profile image uploaded
    private final List<StudentFile> documents = new ArrayList<>();

    public Student(int id, String name, String rollNo, String email, String course) {
        this.id = id; this.name = name; this.rollNo = rollNo; this.email = email; this.course = course;
    }
    public int getId()                        { return id; }
    public String getName()                   { return name; }
    public String getRollNo()                 { return rollNo; }
    public String getEmail()                  { return email; }
    public String getCourse()                 { return course; }
    public int getProfileFileId()             { return profileFileId; }
    public void setProfileFileId(int id)      { this.profileFileId = id; }
    public List<StudentFile> getDocuments() { return documents; }
}
```

StudentFile.java

```
package com.kpr.portal;

/** Metadata of one uploaded file (a row of student_files). */
public class StudentFile {
    private final int id;
    private final String fileType, originalName, mimeType, uploadedAt;
    private final long fileSize;

    public StudentFile(int id, String fileType, String originalName, String mimeType,
                       long fileSize, String uploadedAt) {
        this.id = id; this.fileType = fileType; this.originalName = originalName;
        this.mimeType = mimeType; this.fileSize = fileSize; this.uploadedAt = uploadedAt;
    }
    public int getId()               { return id; }
    public String getFileType()      { return fileType; }
    public String getOriginalName() { return originalName; }
    public String getMimeType()      { return mimeType; }
    public String getUploadedAt()    { return uploadedAt; }
    public long getSizeKb()          { return Math.max(1, fileSize / 1024); }
    public boolean isImage()         { return mimeType != null && mimeType.startsWith("image/"); }
}
```

LoginServlet.java and LogoutServlet.java

```
package com.kpr.portal;

import java.io.IOException;
import java.sql.*;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.sendRedirect(req.getContextPath() + "/login.jsp");
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        String user = req.getParameter("username");
        String pass = req.getParameter("password");
        if (user == null || user.trim().isEmpty() || pass == null || pass.isEmpty()) {
            fail(req, res, "Username and password are required.");
            return;
        }
        // hash is compared inside MySQL, so the plain password is never stored or logged
        String sql = "SELECT id FROM users WHERE username = ? AND password_hash = SHA2(?, 256)";
        try (Connection c = DBUtil.getConnection(); PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setString(1, user.trim());
            ps.setString(2, pass);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    HttpSession old = req.getSession(false);
                    if (old != null) old.invalidate();              // new session id => no session fixation
                    HttpSession session = req.getSession(true);
                    session.setAttribute("user", user.trim());      // marks the session as logged in
                    session.setMaxInactiveInterval(15 * 60);
                    res.sendRedirect(req.getContextPath() + "/students");
                } else {
                    fail(req, res, "Invalid username or password.");
                }
            }
        } catch (SQLException e) {
            throw new ServletException("Login failed", e);
        }
    }

    private void fail(HttpServletRequest req, HttpServletResponse res, String msg)
            throws ServletException, IOException {
        req.setAttribute("error", msg);
        req.getRequestDispatcher("/login.jsp").forward(req, res);
    }
}

// ---- LogoutServlet.java ----
@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        HttpSession session = req.getSession(false);
        if (session != null) session.invalidate();                  // destroy the session
        res.sendRedirect(req.getContextPath() + "/login.jsp");
    }
}
```

AuthFilter.java (session check for protected URLs)

```
package com.kpr.portal;

import java.io.IOException;
import javax.servlet.*;
import javax.servlet.annotation.WebFilter;
import javax.servlet.http.*;

/** Runs before /students and /file: only requests carrying a logged-in session pass. */
@WebFilter(urlPatterns = {"/students", "/file"})
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);               // do not create a session here
        boolean loggedIn = session != null && session.getAttribute("user") != null;

        if (loggedIn) {
            res.setHeader("Cache-Control", "no-store, no-cache, must-revalidate");  // Back button safe
            chain.doFilter(request, response);                     // allowed
        } else {
            res.sendRedirect(req.getContextPath() + "/login.jsp"); // not logged in -> login page
        }
    }
}
```

StudentListServlet.java (JDBC + forward to JSP)

```
package com.kpr.portal;

import java.io.IOException;
import java.sql.*;
import java.util.*;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/students")
public class StudentListServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        // LinkedHashMap keeps students in roll-number order and gives O(1) lookup by id
        Map<Integer, Student> students = new LinkedHashMap<>();

        try (Connection c = DBUtil.getConnection()) {
            // 1) all students
            try (PreparedStatement ps = c.prepareStatement(
                     "SELECT id, name, roll_no, email, course FROM students ORDER BY roll_no");
                 ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {                                          // looping over rows
                    students.put(rs.getInt("id"), new Student(rs.getInt("id"), rs.getString("name"),
                            rs.getString("roll_no"), rs.getString("email"), rs.getString("course")));
                }
            }
            // 2) metadata (path info) of every uploaded file, attached to its student
            try (PreparedStatement ps = c.prepareStatement(
                     "SELECT id, student_id, file_type, original_name, mime_type, file_size, uploaded_at "
                   + "FROM student_files ORDER BY uploaded_at");
                 ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    Student s = students.get(rs.getInt("student_id"));
                    if (s == null) continue;                                 // conditional: orphan row
                    if ("profile".equals(rs.getString("file_type"))) {
                        s.setProfileFileId(rs.getInt("id"));                 // used by <img src="file?id=..">
                    } else {
                        s.getDocuments().add(new StudentFile(rs.getInt("id"), rs.getString("file_type"),
                                rs.getString("original_name"), rs.getString("mime_type"),
                                rs.getLong("file_size"), rs.getString("uploaded_at")));
                    }
                }
            }
        } catch (SQLException e) {
            throw new ServletException("Could not load students", e);
        }

        req.setAttribute("students", new ArrayList<>(students.values()));
        // JSP is under WEB-INF, so it can only be reached through this servlet
        req.getRequestDispatcher("/WEB-INF/views/students.jsp").forward(req, res);
    }
}
```

FileServlet.java (streams images / documents)

```
package com.kpr.portal;

import java.io.IOException;
import java.io.OutputStream;
import java.nio.file.*;
import java.sql.*;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/file")
public class FileServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        int id;
        try { id = Integer.parseInt(req.getParameter("id")); }
        catch (NumberFormatException e) { res.sendError(HttpServletResponse.SC_BAD_REQUEST); return; }

        // the file location comes from the DATABASE, never from the request
        String stored = null, mime = null, original = null;
        try (Connection c = DBUtil.getConnection();
             PreparedStatement ps = c.prepareStatement(
                 "SELECT file_path, mime_type, original_name FROM student_files WHERE id = ?")) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    stored = rs.getString("file_path");
                    mime = rs.getString("mime_type");
                    original = rs.getString("original_name");
                }
            }
        } catch (SQLException e) {
            throw new ServletException("Could not read file metadata", e);
        }
        if (stored == null) { res.sendError(HttpServletResponse.SC_NOT_FOUND); return; }

        Path base = Paths.get(getServletContext().getInitParameter("UPLOAD_DIR"))
                         .toAbsolutePath().normalize();
        Path file = base.resolve(stored).normalize();
        if (!file.startsWith(base) || !Files.isRegularFile(file)) {        // blocks ../ path traversal
            res.sendError(HttpServletResponse.SC_NOT_FOUND);
            return;
        }

        res.setContentType(mime);                                          // image/jpeg, application/pdf ...
        res.setContentLengthLong(Files.size(file));
        res.setHeader("X-Content-Type-Options", "nosniff");
        res.setHeader("Content-Disposition",
                      "inline; filename=\"" + original.replaceAll("[^A-Za-z0-9._-]", "_") + "\"");
        try (OutputStream out = res.getOutputStream()) {
            Files.copy(file, out);                                         // stream bytes to the browser
        }
    }
}
```

login.jsp

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0"><title>Portal Login</title>
<style>
  body { font-family:"Segoe UI",Arial,sans-serif; background:#eef2ff; display:flex;
         justify-content:center; padding-top:80px; }
  form { background:#fff; padding:28px; border-radius:12px; width:300px; box-shadow:0 6px 18px rgba(0,0,0,.12); }
  input { width:100%; padding:10px; margin:6px 0 14px; box-sizing:border-box;
          border:1px solid #d1d5db; border-radius:6px; }
  button { width:100%; padding:10px; background:#1d4ed8; color:#fff; border:0; border-radius:6px; }
  .err { color:#b91c1c; margin-bottom:10px; }
</style></head><body>
<form action="${pageContext.request.contextPath}/login" method="post">
  <h2>Student Portal</h2>
  <c:if test="${not empty error}"><div class="err"><c:out value="${error}"/></div></c:if>
  <label>Username</label><input name="username" required>
  <label>Password</label><input type="password" name="password" required>
  <button>Login</button>
</form>
</body></html>
```

WEB-INF/views/students.jsp

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<c:set var="ctx" value="${pageContext.request.contextPath}"/>
<!DOCTYPE html>
<html lang="en"><head><meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0"><title>Student Profiles</title>
<style>
  body { font-family:"Segoe UI",Arial,sans-serif; background:#eef2ff; margin:0; padding:20px; }
  header { display:flex; justify-content:space-between; align-items:center; flex-wrap:wrap; }
  .grid { display:grid; grid-template-columns:repeat(auto-fill,minmax(260px,1fr)); gap:16px; }
  .card { background:#fff; padding:16px; border-radius:12px; box-shadow:0 4px 12px rgba(0,0,0,.1); }
  .avatar { width:110px; height:110px; border-radius:50%; object-fit:cover; border:3px solid #1d4ed8; }
  .none { display:flex; align-items:center; justify-content:center; background:#e5e7eb; color:#6b7280; }
  .thumb { max-width:70px; max-height:50px; vertical-align:middle; border:1px solid #d1d5db; }
  ul { padding-left:18px; } .muted { color:#6b7280; list-style:none; margin-left:-18px; }
</style></head><body>

<header>
  <h1>Student Profiles</h1>
  <span>Logged in as <b><c:out value="${sessionScope.user}"/></b> |
        <a href="${ctx}/logout">Logout</a></span>
</header>

<c:choose>
  <c:when test="${empty students}"><p>No students found.</p></c:when>
  <c:otherwise>
    <p>Total students: ${students.size()}</p>
    <div class="grid">
      <%-- loop over every student --%>
      <c:forEach var="s" items="${students}" varStatus="st">
        <div class="card">
          <small>#${st.count}</small>
          <%-- conditional: photo (streamed by FileServlet) or placeholder --%>
          <c:choose>
            <c:when test="${s.profileFileId > 0}">
              <img class="avatar" src="${ctx}/file?id=${s.profileFileId}" alt="Profile photo">
            </c:when>
            <c:otherwise><div class="avatar none">No photo</div></c:otherwise>
          </c:choose>
          <h3><c:out value="${s.name}"/></h3>
          <div>Roll No: <c:out value="${s.rollNo}"/></div>
          <div>Course: <c:out value="${s.course}"/></div>
          <div>Email: <c:out value="${s.email}"/></div>

          <b>Documents</b>
          <ul>
            <c:if test="${empty s.documents}"><li class="muted">No documents uploaded</li></c:if>
            <%-- loop over this student's files --%>
            <c:forEach var="d" items="${s.documents}">
              <li>
                <c:out value="${d.fileType}"/>:
                <a href="${ctx}/file?id=${d.id}" target="_blank"><c:out value="${d.originalName}"/></a>
                (${d.sizeKb} KB)
                <c:if test="${d.image}"><br><img class="thumb" src="${ctx}/file?id=${d.id}" alt="document"></c:if>
              </li>
            </c:forEach>
          </ul>
        </div>
      </c:forEach>
    </div>
  </c:otherwise>
</c:choose>
</body></html>
```

Output Screenshots

Paste the browser screenshots below after running the project (each with the URL bar visible).

1. register.html - empty form, validation errors, password strength meter, valid state

2. php/index.php - student list after adding 2-3 students

3. php/add.php - server-side error (wrong file type / file too large / duplicate roll number)

4. php/view.php - student details with profile image and uploaded documents table

5. php/edit.php and delete confirmation, phpMyAdmin view of student_files table

6. Servlet login.jsp (wrong password message) and redirect to login when URL /students is opened without login

7. Servlet /students - profile cards with images streamed by /file?id=..., document links, logout

Conclusion

The assignment delivers a complete student admission workflow. A responsive HTML5/CSS3 registration form with JavaScript validation gives immediate feedback, while PHP and MySQL repeat the checks on the server and provide full CRUD with safe file uploads (type, size and no-overwrite rules) whose metadata is linked to each student. A Servlet/JSP portal then reads the same database through JDBC, streams the stored files, and restricts access with session-based login. Prepared statements, output escaping, unique file names and path checks make the system secure as well as functional.

Repository

GitHub repository link: `https://github.com/<your-username>/student-portal` (replace with your link; include `student_portal/`, `StudentPortalWeb/`, `db/schema.sql` and the screenshots).